Author's Note — Chapter 19

Building Production Prompt Systems: From Prototype to Pipeline

Tanya Bansal | INFO 7375: Prompt Engineering for Generative AI | Northeastern University

---

Page 1: Design Choices

Why This Chapter

I chose Chapter 19 — productionizing prompt systems — because it sits at the intersection of what I already know and what I needed to learn. My professional background at Jio Platforms Limited involved building and maintaining CI/CD pipelines, containerized microservices, and scheduler-driven infrastructure automation across enterprise-scale systems. I spent four years learning, sometimes painfully, that the gap between "it works on my machine" and "it works in production at 2 AM on a Saturday" is almost entirely architectural. The code is the same. The infrastructure around it is what determines whether a failure is a five-minute rollback or a three-hour incident.

When I started building LLM-powered features — most recently my Job Application Portal project, which integrates a React/TypeScript frontend with a Spring Boot backend, an Oracle database, Redis caching, JWT authentication, and a RAG pipeline using the OpenAI GPT API for candidate-job matching, all orchestrated in a multi-container Docker environment — I noticed that I was repeating the same architectural mistakes the industry had already solved for traditional software. My prompts were hardcoded strings. My LLM outputs were trusted implicitly. I had no evaluation loop, no fallback, and no observability for the prompt layer. I chose this chapter because I wanted to formalize the production patterns I was already learning through trial and error.

The Architectural Pattern

The chapter introduces a "Five Walls" architecture for production prompt systems: a Prompt Registry (versioning and rollback), a Schema Gate (structural output validation), an Evaluation Loop (semantic drift detection), a Fallback Cascade (graceful degradation), and an Observability Layer (full execution tracing). The central argument is that each wall blocks a distinct failure class that the other four cannot catch. The walls are orthogonal defenses, not redundant layers.

What I Left Out

I deliberately excluded multi-model routing, A/B testing at scale (beyond the basic traffic-weighted routing implemented in the Prompt Registry), and cost optimization. Each deserves its own chapter. Including them would have diluted the core argument — that the five walls are independently necessary — into a survey of production concerns. I also excluded real-world LLM API integration in favor of a mock, which keeps the notebook runnable without API keys but

means the reader does not experience actual API latency variability or real model output stochasticity.

Connection to the Master Argument

The book's master argument is: *architecture is the leverage point, not the model.* My chapter is a specific instance of this claim. The same GPT-4 model, given the same prompt, produces reliable or unreliable results depending entirely on whether the pipeline architecture includes an evaluation gate and fallback logic. The failure experiments in Section 4 prove this empirically: four times, we inject the same failure into the same model, and the outcome depends entirely on which architectural wall is present or absent. The model does not change. The architecture does.

Page 2: Tool Usage

Bookie the Bookmaker — Prose Generation

I used Bookie to generate the initial chapter draft, providing a detailed outline with Tetrahedron mappings (Structure, Logic, Implementation, Outcome for each section) and the mandatory section ordering (Scenario → Mechanism → Design Decision → Failure Case → Exercise). Bookie produced a strong first draft with a vivid opening scenario, the five-wall metaphor, and mechanistic failure experiments.

Correction 1 — Missing Pipeline Integration. Bookie generated five individual wall descriptions but never showed how the walls wire together as a system. The PromptRegistry, SchemaGate, EvaluationLoop, FallbackCascade, and ObservabilityLayer were presented as standalone classes. I added a `PromptPipeline` class that composes all five and added prose explaining the inter-wall signal flow: the Schema Gate's `SchemaViolationError` exception propagates into the Fallback Cascade's `try/except`, the Evaluation Loop scores only post-validation output, and the Observability Layer wraps the entire chain with `start_trace()` and `complete_trace()`.

Correction 2 — Black-Box Evaluation Scoring. Bookie's draft included a `_compute_similarity` method with no explanation of what similarity means. I replaced this with an explicit two-signal explanation: field-level exact match for constrained fields (sentiment, priority) and embedding cosine similarity for freeform fields (issue).

Correction 3 — Repeated Thesis. The Section 4 conclusion repeated the Section 2 thesis nearly verbatim ("orthogonal defenses... each one catches something the others structurally cannot"). I rewrote the Section 4 conclusion to build on the experimental evidence: the Schema Gate and Evaluation Loop both examine output but are blind to each other's failure class; the Fallback Cascade and Prompt Registry both handle recovery but on different timescales.

Eddy the Editor — Pedagogical Audit

Eddy audited the revised chapter across five dimensions (Feynman Standard, Architecture-Without-Mechanism, Tetrahedron Completeness, Sycophantic AI Usage, and Exercise Quality)

and rated it 8/10 for Substack readiness with six priority fixes. The most impactful findings and my responses:

Correction 4 — Cosine Similarity Stops Short of Feynman (Eddy Soft Spot 1). Eddy flagged that the cosine similarity explanation tells the reader what it measures but not *why* vector angle corresponds to semantic closeness. I added a full paragraph explaining that embedding models are trained to push similar texts to nearby points in vector space, and that cosine measures directional alignment regardless of magnitude — two descriptions of the same login bug point the same way even if one is longer.

Correction 5 — Cache Lookup Mechanism Undefined (Eddy Soft Spot 2). Eddy flagged that the cache uses input text hashing, meaning "Login is broken!!!" and "Login hasn't worked in 3 days" would never match. I rewrote the cache section to use embedding-based similarity lookup rather than exact hash matching, connecting it back to the same embedding mechanism from Section 3.3.

Correction 6 — Exercise Step 4 Needs Scaffolding (Eddy Critical Flag). Eddy's strongest criticism was that the exercise's writing prompt could be answered with a vague restatement of the section header. Eddy proposed a four-constraint scaffold requiring the student to: (1) name a specific failure from Step 3, (2) explain why the schema gate cannot catch it, (3) explain how the evaluation loop catches it, and (4) state the business consequence. I adopted this scaffold and also added a Step 5 extension exercise about medical triage defaults, connecting to the Human Decision Node theme.

Correction 7 — Duplicate Pipeline-vs-Checklist Framing (Eddy Sycophantic AI Flag 1). Eddy flagged that the paragraph "Five walls, described individually, are still just five concepts..." duplicated the post-code synthesis. I deleted the pre-code version and let the post-code explanation earn the synthesis by referencing specific methods.

Correction 8 — Observability Soundbite Duplication (Eddy Sycophantic AI Flag 2). Eddy flagged that "Observability is not logging" appeared as a soundbite in Section 2 and was then re-derived mechanistically in Section 3.5. I kept the Section 2 description focused on what observability *does* and saved the "not logging" distinction for Section 3.5 where it's earned through the structured-log example.

Correction 9 — Bumper-Sticker Final Takeaway (Eddy Sycophantic AI Flag 3). Eddy flagged that "Architecture is the leverage point. The model is just what executes the architecture you designed." is a slogan, not a takeaway. I replaced it with an operational statement: "When a production prompt system fails, the diagnostic question is never 'what's wrong with the prompt?' — it's 'which wall is missing or misconfigured?' The five-wall framework gives you five specific places to look."

Eddy Flag I Rejected — A/B Testing Claim. Eddy flagged that the Prompt Registry claims A/B testing but the code only implements `fetch()` and `rollback()`. Rather than downgrading the claim as Eddy suggested, I implemented A/B routing with a `traffic_weight` field in the YAML schema and `random.choices` weighted selection in the `fetch()` method, closing the gap Eddy

identified. The notebook uses simplified highest-active-version routing with a comment noting the chapter's full implementation.

Figure Architect — Visualization

Figure Architect scanned the chapter and identified 7 high-assertion zones requiring figures, prioritized by mechanism complexity and verification gaps. The three critical-priority figures were: (1) the Five-Wall Pipeline Architecture showing inter-wall signal flow, (2) the Three-Panel Experiment Comparison showing identical outputs with different diagnostic results, and (3) the Wall-Removal Failure Matrix as a visually encoded evidence table. Figure Architect also identified two zones I had not considered: a Dual-Signal Evaluation diagram showing how exact match and cosine similarity feed a combined score, and an Orthogonal Defense Coverage Matrix (5×5 grid with diagonal pattern proving each wall blocks exactly one failure class). I generated all 8 figures (7 content figures + 1 hero image) and integrated them into the Substack article at the positions Figure Architect recommended.

Courses — Learning Outcomes & Lesson Sequence

I ran the chapter's core claim and audience description through Courses, which generated five Bloom's-aligned outcomes at levels 3–6 (Apply, Analyze, Analyze, Evaluate, Create). Each outcome included a specific assessment anchor tied to the chapter's actual content — for example, Outcome 2 (Evaluate) anchors to the Human Decision Node, requiring students to critique the AI's proposed 24-hour cache TTL by identifying the domain-specific risk. The Courses outcomes validated my existing set — both cover the same cognitive range with no outcomes at Remember or Understand level — so I kept my original wording to maintain consistency across the chapter, notebook, and video. Courses explicitly noted: "Any quiz item that asks 'List the Five Walls' is testing recall, not competence" — which confirmed my chapter's design philosophy of teaching through failure experiments rather than taxonomy.

Eddy the Storyboarder — Video Architecture

I pasted the final chapter into Eddy the Storyboarder with a 10-minute target length. Eddy the Storyboarder generated a complete scene-by-scene CapCut storyboard following the Explain → Show → Try structure, with specific narration text, on-screen text headlines, visual direction, and transition instructions for each scene. The storyboard confirmed my planned video structure and provided precise timing breakdowns: the Explain act (2:30) states the architectural claim with the prototype-vs-production diagram, the Show act (5:30) walks through the notebook with the Human Decision Node as the centerpiece moment, and the Try act (2:00) gives the viewer the evaluation-loop toggle exercise. I used the storyboard as my recording guide.

Human Decision Node — AI Scaffold (GPT-4.1)

The AI scaffold was a real GPT-4.1 (ChatGPT) call that proposed a complete pipeline configuration given a description of the customer support ticket summarization system. The AI's proposal was comprehensive — it included evaluation thresholds, fallback ordering with a schema_repair tier, cache TTL, default values, confidence routing controls, PII redaction guardrails, and SLA parameters. Three rejections and two acceptances:

Correction 10 — Cache TTL of 24 Hours. The AI proposed 86,400 seconds (24 hours). I rejected this and reduced it to 1,800 seconds (30 minutes). Support tickets are time-sensitive — a cached summary from 24 hours ago could reference a since-resolved issue.

Correction 11 — Evaluation Rubric Weights. The AI proposed a quality threshold of 0.90 with only 0.20 weight on priority accuracy. I rejected the weighting because the priority field directly controls ticket routing — a miscalibrated priority has a far higher cost of error than a miscalibrated sentiment (which is analytics-only). I adjusted priority weight to 0.40 and lowered the overall threshold to 0.85 to reduce false positives under the corrected weighting.

Correction 12 — Keyword Overrides in Deterministic Default. The AI proposed keyword-based priority overrides in the fallback default tier (e.g., "outage" → High). I rejected this because the deterministic default's value is its predictability. Adding keyword matching introduces a new failure surface in the safety net — the one component that must never fail. Keyword intelligence belongs in the primary LLM path, not in the last-resort fallback.

Accepted — Inline Evaluation Mode. The AI proposed inline evaluation. I agreed — for a priority-routing system, batch evaluation that discovers a misrouted HIGH-priority ticket 30 minutes later is too slow.

Accepted — Schema Repair Tier. The AI proposed a "schema_repair" tier before retry in the fallback ordering. I accepted this as a useful addition I had not considered — it could reduce unnecessary fallback cascading for simple format issues.

Page 3: Self-Assessment

Rubric Scoring

Architectural Rigor (33/35). The five-wall architecture is well-defined, and each wall's failure class is mechanistically traced. The four failure experiments provide empirical proof that each wall is independently necessary. The pipeline integration code shows how the walls compose into a functioning system. The A/B testing claim is now backed by an implementation with `traffic_weight` and `random.choices`. Where I lose points: the chapter could benefit from a more explicit discussion of wall interaction failure modes — what happens when two walls conflict (e.g., the evaluation loop flags drift but the fallback cascade serves a cached response that also has the drifted pattern).

Technical Implementation (23/25). The Jupyter notebook is fully runnable from a fresh clone with no API keys required. All five walls are implemented as working Python classes. The failure experiments are triggerable — the reader can toggle walls on and off with feature flags. The AI scaffold uses a real GPT-4.1 output with three documented rejections and two acceptances. Where I lose points: the evaluation loop uses Jaccard token overlap as a similarity proxy instead of actual embedding cosine similarity. The notebook comments clearly mark this as a demo simplification with the production implementation documented in the chapter prose.

Pedagogical Clarity (19/20). The Feynman Standard is maintained throughout — every concept is introduced through the scenario's failure before it gets a name. The "Five Walls" metaphor provides a memorable structural framework. The exercise has a deliberate pedagogical trap (students who only build schema validation discover it cannot catch semantic drift) and now includes Eddy's four-constraint scaffold for Step 4 plus a Step 5 medical triage extension. The cosine similarity explanation now goes a full layer deeper into embedding intuition, addressing Eddy's Feynman compliance flag. Where I lose points: the observability section could still benefit from a more concrete step-by-step post-incident query walkthrough.

Relative Quality (17/20). The chapter reads like a professional technical reference with 8 publication-quality figures integrated at architecturally significant positions. The Human Decision Node is visible in the notebook (real GPT-4.1 output with three rejections), documented in this Author's Note, and demonstrated on camera in the video. All five professor-provided tools (Bookie, Eddy, Figure Architect, Courses, Eddy the Storyboarder) were used and their outputs informed specific revisions. Where I lose points: the failure experiments use mock data. A more compelling demonstration would use the actual OpenAI API with real model stochasticity.

Total Self-Assessment: 92/100

What Could Be Improved

The failure mode in my demo is mechanistically correct — removing the evaluation gate allows semantically degraded outputs to pass through every other check undetected. But the demo operates on a fixed set of five test tickets with a mock LLM that returns from a small pool of predetermined responses. In a real production system, the failure would compound with input distribution drift: as the types of support tickets change over time, even a stable model with a stable prompt can produce degrading outputs because the input distribution has shifted away from what the golden set represents.

Adding a synthetic input drift generator — a function that gradually shifts the test ticket distribution away from the golden set's coverage — would make the failure case significantly more realistic. It would also introduce a sixth failure class that the current five walls do not fully address: golden set staleness. The evaluation loop detects drift relative to the golden set, but it cannot detect when the golden set itself has become unrepresentative. This is an architectural gap my chapter identifies in the video's closing question but does not resolve — and resolving it would require a golden set refresh mechanism that is, itself, a production operational practice worthy of its own section.